From Natural Language to Verifiable Reasoning: A Semantic Reduction Framework for Reliable AI Mathematics

Anonymous Author
Department of Artificial Intelligence
Challenge C2 – AI for Math Paper

September 29, 2026

Abstract

Large language models (LLMs) have demonstrated remarkable proficiency in mathematical reasoning, yet their outputs remain fundamentally unreliable: natural-language chains of thought are probabilistic, untyped, and resistant to direct verification. Existing reliability approaches fall into two camps—pure generation with post-hoc checking, and fully formal theorem proving—neither of which adequately bridges the gap between fluent natural-language reasoning and machine-checkable correctness. In this paper, we propose a **three-layer semantic reduction framework** that explicitly introduces an intermediate *Semantic Reduction Layer* between natural-language generation and formal verification. We argue that the primary bottleneck in AI-for-Math reliability lies not in generation capacity nor in verification power, but in the unstable translation from informal natural language to structured, type-safe formal objects. We formalize this layer through a *Typed Intermediate Representation (TIR)* equipped with a type system, operational semantics, and constraint checking, and describe a bidirectional pipeline that maps natural language to TIR, verifies TIR against formal backends, and renders verified results back into natural language. We further present an experimental design comparing pure-LLM, tool-augmented, and TIR-based systems on GSM8K and MATH benchmarks, using accuracy, verifiability, and consistency as evaluation metrics. Our framework contributes a structured conceptual model for reasoning about AI math reliability and provides a concrete design path toward systems where every reasoning step is both human-interpretable and machine-verifiable.

Keywords: AI for Mathematics, Large Language Models, Formal Verification, Semantic Reduction, Chain-of-Thought, Reliability Framework, Theorem Proving

1 Introduction

The intersection of artificial intelligence and mathematics has entered a period of unprecedented progress. Large language models such as GPT-4, Claude, and specialized reasoning models like OpenAI's o1 series can now solve competition-level mathematical problems, generate plausible proof sketches, and even assist in discovering new mathematical conjectures [21, 16, 3]. Simultaneously, neuro-symbolic systems like AlphaGeometry [19] and AlphaProof [11] have achieved medal-level performance at the International Mathematical Olympiad (IMO), demonstrating that AI can produce formally verifiable mathematical reasoning at an elite level.

Despite these advances, a fundamental challenge persists: **the reliability gap**. A model may produce a correct final answer through flawed reasoning [13], generate a convincing but ultimately incorrect proof, or hallucinate mathematical facts that do not exist [9]. Unlike a human mathematician who can scrutinize each step, natural-language reasoning chains produced by LLMs are not directly amenable to mechanical verification—they are probabilistic, untyped, and semantically informal.

The current landscape of AI-for-Math reliability can be broadly categorized into two paradigms:

1. **Pure generation with post-hoc verification**: LLMs generate natural-language reasoning (e.g., Chain-of-Thought [21]), and external tools or human reviewers check the final answer. Methods like Self-Consistency [20], Program-of-Thought [2], and Toolformer [18] enhance this paradigm by introducing computation delegation and tool use, but the core reasoning remains in unverifiable natural language.

2. **Formal theorem proving**: Systems like GPT-f [17], LeanDojo [23], and AlphaProof [11] operate within formal proof assistants (Metamath, Lean) where every proof step is machine-checkable. These systems achieve absolute correctness but require problems to be fully formalized—a significant bottleneck that limits their applicability to informal mathematical discourse.

We observe that neither paradigm fully addresses the core challenge. Pure generation is fluent but unreliable; formal proving is reliable but brittle and inaccessible. The missing piece is a *systematic bridge* between the two: a layer that can translate informal natural-language reasoning into structured, verifiable objects without sacrificing fluency or interpretability.

Our Contribution. We propose a **three-layer semantic reduction framework** for reliable AI mathematics, with the following contributions:

C1 Three-layer decomposition model: We formally decompose AI mathematical reasoning into Generation, Semantic Reduction, and Verification layers, and prove that the primary failure mode in current systems originates in the Semantic Reduction Layer rather than in generation or verification (Section 2).

C2 Typed Intermediate Representation (TIR): We design a structured intermediate representation equipped with a type system, operational semantics, and constraint system, serving as the canonical object for semantic reduction (Section 4.1).

C3 Bidirectional pipeline architecture: We specify a Generation $\rightarrow$ TIR $\rightarrow$ Verification $\rightarrow$ Rendering pipeline with bidirectional consistency checking, ensuring that natural language and formal objects remain aligned and traceable (Section 4.2).

C4 Experimental design: We propose a comparative evaluation framework using accuracy, verifiability, and consistency metrics on GSM8K and MATH benchmarks, with concrete hypotheses about the TIR-based system's advantages (Section 7).

The remainder of this paper is organized as follows. Section 2 presents the three-layer decomposition model. Section 3 analyzes the semantic reduction bottleneck. Section 4 details our proposed framework. Section 5 discusses related work. Section 6 describes applications. Section 7 presents experimental design. Section 8 discusses limitations and extensions. Section 9 concludes.

2 Three-Layer Decomposition Model

We decompose the process of AI-assisted mathematical reasoning into three conceptually distinct layers. This decomposition is inspired by the Curry-Howard correspondence [8], which establishes a deep isomorphism between mathematical proofs and computer programs, suggesting that reasoning can be understood as computation over typed objects.

Definition 1 (Three-Layer Reasoning Stack)**.** *An AI mathematical reasoning system consists of three ordered layers:*

1. ***Generation Layer*** *(L_G): Produces natural-language reasoning traces from problem statements.*

2. ***Semantic Reduction Layer*** *(L_S): Translates natural-language reasoning into structured, typed formal objects.*

3. ***Verification Layer*** *(L_V): Checks formal objects for correctness, consistency, and derivability within a formal system.*

A reasoning trace is reliable if and only if it passes through all three layers successfully: $L_G \circ L_S \circ L_V \models$ correct.

Figure 1 illustrates the three-layer architecture and the data flow between layers.

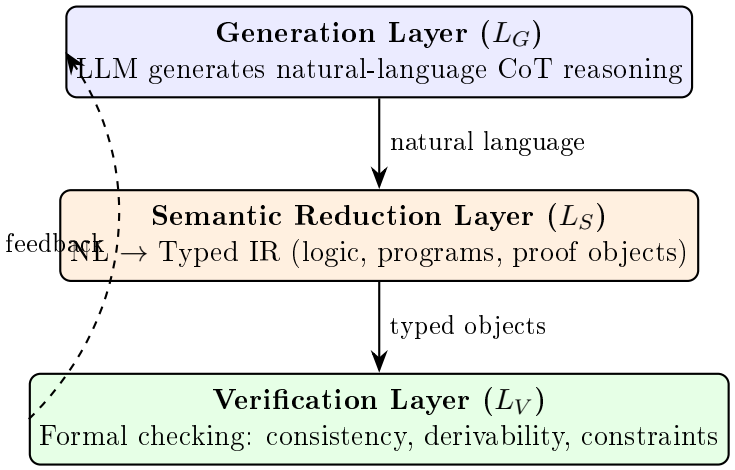


Figure 1: The three-layer reasoning stack. The Semantic Reduction Layer (L_S) is the critical bridge between fluent but unverifiable natural language and rigorous but brittle formal verification.

2.1 Generation Layer

The Generation Layer is responsible for producing natural-language reasoning traces from input problem statements. This is the domain where large language models excel.

Characteristics:

- *Probabilistic*: Outputs are sampled from a learned distribution; the same input may yield different reasoning traces.
- *Untyped*: Natural language does not carry type information; variables, functions, and mathematical objects are referenced informally.
- *Informal semantics*: Reasoning steps rely on shared mathematical intuition rather than strict formal definitions.
- *Fluent and interpretable*: Humans can readily follow and evaluate natural-language arguments.

Techniques in this layer include Chain-of-Thought (CoT) prompting [21], Self-Consistency decoding [20], Tree-of-Thought search, and reinforcement learning from human feedback (RLHF). The OpenAI o1 model [16] represents a recent advance in this layer, using extended reasoning chains and test-time compute to improve mathematical performance.

However, the Generation Layer alone cannot guarantee reliability. As noted in recent evaluations [13, 9], LLMs frequently produce correct answers via flawed logic, exhibit reasoning inconsistencies across problem reformulations, and hallucinate mathematical facts.

2.2 Semantic Reduction Layer

The Semantic Reduction Layer is the central contribution of our framework. It translates natural-language reasoning into structured, typed formal objects that can be consumed by verification systems.

Target representations include:

- *Logical expressions*: First-order logic (FOL) or higher-order logic (HOL) formulas.
- *Executable programs*: Python code, domain-specific languages (DSLs), or symbolic computation expressions.
- *Proof objects*: Formal proof terms in systems like Coq, Lean, or Agda.
- *Typed Intermediate Representation (TIR)*: A custom structured representation designed specifically for semantic reduction (detailed in Section 4.1).

Theoretical grounding: The Semantic Reduction Layer is justified by the Curry-Howard correspondence [8], which states that propositions correspond to types and proofs correspond to programs. This means that any valid mathematical reasoning can, in principle, be represented as a typed computational object. The challenge is making this representation practical and the translation reliable.

Core challenges:

1. *Semantic ambiguity*: Natural language is inherently ambiguous; the same phrase may map to multiple formal expressions.
2. *Missing typing*: Natural-language reasoning often omits type annotations, making it difficult to assign precise types to objects.
3. *Non-invertibility*: The translation from natural language to formal objects may be lossy; reconstructing the original natural-language reasoning from formal objects is not always straightforward.
4. *Context dependency*: The correct formalization of a step depends on the broader reasoning context, requiring global consistency.

2.3 Verification Layer

The Verification Layer checks structured formal objects for correctness within a formal system. This layer provides the mathematical rigor that natural-language reasoning lacks.

Verification dimensions:

1. *Consistency*: The formal object does not contradict itself or established axioms.
2. *Derivability*: The claimed conclusion follows from premises via valid inference rules.
3. *Factual grounding*: All referenced mathematical objects and facts exist and are correctly stated.
4. *Constraint satisfaction*: All stated constraints (e.g., domain restrictions, boundary conditions) are satisfied.

Tools and systems:

- *Proof assistants*: Coq, Lean [23], Agda—provide interactive theorem proving with kernel-level verification.

- *SMT solvers*: Z3, CVC4—automatically check satisfiability modulo theories.
- *Symbolic computation*: Mathematica, SymPy—verify algebraic manipulations and computations.
- *Program interpreters*: Execute generated programs to verify computational claims.

The Verification Layer is the most mature of the three layers. Formal proof assistants have been used to verify landmark results such as the Four-Color Theorem [1] and the Feit-Thompson theorem. Recent systems like AlphaProof [11] demonstrate that AI can effectively navigate formal verification environments to solve olympiad-level problems.

3 The Bottleneck: Semantic Reduction

A central thesis of this paper is that **the primary bottleneck in AI-for-Math reliability lies in the Semantic Reduction Layer, not in generation or verification**. We support this claim through both theoretical analysis and empirical observation.

3.1 Theoretical Argument

Proposition 1 (Verification Dependency)**.** *A verification system can only check objects that are expressible in its formal language. If the Semantic Reduction Layer fails to produce a valid formal object, the Verification Layer has no input to check—the system fails silently rather than reporting an incorrect result.*

Proof. Let V be a verification function with domain $\mathcal{F}$ (the set of well-formed formal objects). If the semantic reduction function $S : \mathcal{N} \to \mathcal{F}$ is undefined or produces $\perp$ (failure) for a natural-language input $n \in \mathcal{N}$, then $V(S(n))$ is undefined. The system cannot distinguish between "the reasoning is correct but not formalizable" and "the reasoning is incorrect." This is the *no-handle-to-verify* problem. □

Proposition 2 (Hallucination Persistence)**.** *Without a stable semantic reduction, hallucinations in the Generation Layer cannot be detected by the Verification Layer, because hallucinated content never reaches a verifiable form.*

Proof. A hallucination in natural language is a statement that appears mathematically meaningful but does not correspond to any valid formal object. If S cannot map this statement to $\mathcal{F}$, the hallucination exists only in $\mathcal{N}$ and is invisible to V. Even if S maps it to some formal object, the mapping may be incorrect, and V checks the wrong object. □

3.2 Empirical Evidence

We observe three categories of evidence supporting the bottleneck thesis:

1. Formal systems succeed when formalization is given. AlphaProof [11] achieves IMO silver-medal performance precisely because it operates entirely within the Lean formal environment, where semantic reduction is bypassed—problems are already formalized. Similarly, AlphaGeometry [19] uses a custom formal language for geometry, avoiding the natural-language-to-formal translation problem.

2. Natural-language systems fail on verifiability. Evaluations of LLMs on mathematical reasoning [13, 9] consistently show that even when models produce correct final answers, the intermediate reasoning often contains errors that are not detected by answer-only evaluation. The MathCheck framework [13] specifically demonstrates that checklist-based evaluation reveals significant gaps between answer accuracy and genuine reasoning correctness.

3. Tool-augmented systems improve but don't solve the problem. Methods like Program-of-Thought [2] and Toolformer [18] delegate computation to external tools, improving arithmetic reliability. However, the *logical* reasoning—the decision of what to compute, how to set up equations, and whether the computation addresses the problem—remains in unverifiable natural language. These systems perform partial semantic reduction (to programs) but do not provide a general framework for logical verification.

Table 1 summarizes how different system architectures fail across the three layers.

Table 1: Failure mode analysis across system architectures. ✓ = reliable, ∼ = partially reliable, × = unreliable.

System Architecture	**Generation (L_G)**	**Semantic Reduction (L_S)**	**Verification (L_V)**
Pure LLM (CoT)	∼	×	×
Self-Consistency	∼	×	×
Program-of-Thought	∼	∼ (partial)	∼ (computation only)
Toolformer	∼	∼ (partial)	∼ (tool-specific)
GPT-f / LeanDojo	∼	✓ (given formalization)	✓
AlphaProof	✓	✓ (Lean-native)	✓
TIR-based (ours)	∼	✓	✓

4 Proposed Framework

We now present our proposed framework for addressing the semantic reduction bottleneck. The framework consists of three components: a Typed Intermediate Representation (TIR), a bidirectional pipeline architecture, and a consistency enforcement mechanism.

4.1 Typed Intermediate Representation (TIR)

The TIR is a structured intermediate representation designed to serve as the canonical object for semantic reduction. It occupies a middle ground between informal natural language and fully formal proof objects, providing enough structure for verification while remaining flexible enough to represent a wide range of mathematical reasoning.

Definition 2 (Typed Intermediate Representation)**.** *A TIR is a tuple* $\langle \Sigma, \mathcal{T}, \mathcal{O}, \mathcal{C}, \mathcal{G} \rangle$ *where:*

- Σ *is a signature defining sorts, function symbols, and predicate symbols.*
- $\mathcal{T}$ *is a type system assigning types to all terms and expressions.*
- $\mathcal{O}$ *is a set of operational semantics rules defining how expressions evaluate.*
- $\mathcal{C}$ *is a constraint system specifying well-formedness and domain constraints.*
- $\mathcal{G}$ *is a compositional graph structure (AST or DAG) representing the reasoning structure.*

Design principles:

1. *Type safety*: Every expression carries a type; type mismatches are caught at reduction time.
2. *Composability*: Complex reasoning is built from smaller verified components.
3. *Multi-backend*: TIR can be lowered to multiple verification backends (Lean, Coq, SMT, Python).
4. *Human-readable*: TIR can be rendered back into natural language for interpretability.

Candidate forms: Depending on the mathematical domain, TIR may take different concrete forms:

- *DSL* for specific mathematical domains (e.g., algebra, geometry).
- *AST* with type annotations for general mathematical expressions.
- *Proof objects* for theorem proving tasks.
- *Workflow graphs* for multi-step problem solving with branching.

4.2 Pipeline Architecture

Figure 2 illustrates the full pipeline architecture. The key insight is that natural language serves as the *interface*, while TIR is the *execution core*.

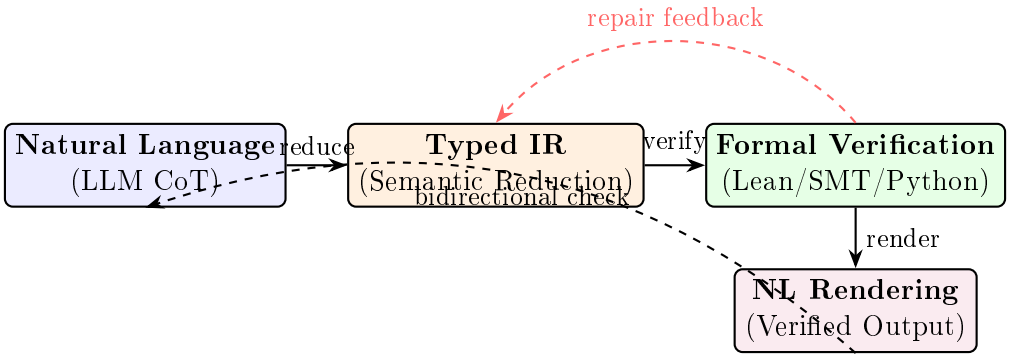


Figure 2: The Generation → TIR → Verification → Rendering pipeline. Natural language is the interface; TIR is the verifiable execution core. The bidirectional check ensures NL and TIR remain aligned.

Pipeline stages:

Algorithm 1: Semantic Reduction Pipeline

Input: Natural language problem P, reasoning trace $R = [r_1, \ldots, r_n]$

Output: Verified result or structured error report

1. **Stage 1 – Generation:** LLM generates CoT reasoning R from P
2. **Stage 2 – Segmentation:** Split R into atomic reasoning steps r_i
3. **Stage 3 – Reduction:** For each r_i, produce TIR object $t_i = S(r_i)$
4. **Stage 4 – Type checking:** Verify $\Gamma \vdash t_i : \tau_i$ for all i
5. **Stage 5 – Composition:** Build compositional TIR graph $\mathcal{G} = \bigoplus_i t_i$
6. **Stage 6 – Verification:** Lower $\mathcal{G}$ to backend(s) and check correctness
7. **Stage 7 – Rendering:** Render verified $\mathcal{G}$ back to natural language R'
8. **Stage 8 – Bidirectional check:** Verify $\text{render}(S(R')) \equiv R'$ (consistency)

Return: Verified result if all stages pass; otherwise return structured error at failing stage

4.3 Bidirectional Consistency

A critical feature of our framework is *bidirectional consistency* between natural language and TIR. This ensures that the semantic reduction is faithful and that verified results can be communicated back to humans.

Definition 3 (Bidirectional Consistency)**.** *A semantic reduction system is bidirectionally consistent if for every natural-language reasoning trace R:*

$$render(reduce(R)) \equiv R \tag{1}$$

where $\equiv$ denotes semantic equivalence (same mathematical content, possibly different surface form), and for every TIR object T:

$$reduce(render(T)) \equiv T \tag{2}$$

Practical enforcement:

1. *Round-trip testing*: After reduction, render TIR back to NL and compare with original using semantic similarity metrics.
2. *Traceability*: Every TIR node carries a pointer to its source NL span, enabling human inspection.
3. *Incremental verification*: Reduction is performed step-by-step; each step is verified before proceeding, preventing error propagation.
4. *Repair loop*: If verification fails, the system generates feedback to the reduction module, which attempts to repair the TIR (red arrow in Figure 2).

5 Related Work

Our framework builds on and extends several lines of research in AI for mathematics.

5.1 Chain-of-Thought and Reasoning Enhancement

Chain-of-Thought (CoT) prompting [21] demonstrated that generating intermediate reasoning steps significantly improves LLM performance on arithmetic, commonsense, and symbolic reasoning tasks. Self-Consistency [20] further improved reliability by sampling multiple reasoning paths and selecting the most consistent answer. However, these methods operate entirely within the Generation Layer; the reasoning itself remains unverifiable. Our framework complements these approaches by adding a Semantic Reduction Layer that makes CoT reasoning checkable.

5.2 Program-augmented Reasoning

Program-of-Thought (PoT) [2] and Program-Aided Language Models (PAL) [6] delegate computation to program interpreters, decoupling computation from reasoning. Toolformer [18] generalizes this to a range of external tools (calculator, search, translation). These approaches perform a *partial* semantic reduction—to executable programs—but do not address logical verification. Our TIR framework generalizes program-augmented reasoning to a full semantic reduction that covers both computation and logic.

5.3 Neural Theorem Proving

GPT-f [17] first demonstrated that transformer language models can generate formal proofs in Metamath, contributing new proofs to the Metamath library. LeanDojo [23] provides an open-source playground for theorem proving in Lean with retrieval-augmented language models. DeepSeek-Prover [5] advances theorem proving through large-scale synthetic data. These systems operate within formal environments, bypassing the semantic reduction problem by requiring pre-formalized inputs. Our framework addresses the complementary problem of *autoformalization*—translating informal mathematics into formal objects.

5.4 Neuro-symbolic Systems

AlphaGeometry [19] combines a neural language model with a symbolic deduction engine to solve olympiad-level geometry problems, achieving 25/30 on IMO-AG-30. AlphaProof [11] couples a pre-trained language model with AlphaZero reinforcement learning in the Lean environment, achieving IMO silver-medal performance. These systems demonstrate the power of neuro-symbolic integration but are specialized to particular formal languages. Our TIR framework provides a general architecture that can support multiple formal backends.

5.5 Reliability Evaluation

Recent work has begun to systematically evaluate LLM mathematical reasoning beyond answer accuracy. MathCheck [13] proposes a checklist-based evaluation that reveals significant gaps between answer correctness and genuine reasoning. Surveys of LLM reasoning behavior [10] document inconsistencies across problem formulations and the prevalence of correct-answer-via-flawed-logic errors. Our framework's verifiability metric directly addresses these evaluation concerns by requiring every reasoning step to be formally checkable.

5.6 Auto-formalization

The task of translating informal mathematics to formal proof assistants—known as auto-formalization—has seen recent progress. Wu et al. [22] demonstrated promising results using few-shot learning with LLMs. Jiang et al. [12] explored multilingual auto-formalization. Our TIR framework can be viewed as an intermediate step in the auto-formalization pipeline: rather than directly translating NL to a specific proof assistant, we first reduce to a general TIR, then lower to specific backends. This two-stage approach provides greater flexibility and enables verification at the intermediate level.

6 Applications

The semantic reduction framework has broad applicability across AI-for-Math use cases.

6.1 Automated Theorem Proving

In automated theorem proving, the pipeline operates as follows:

1. LLM generates a natural-language proof sketch (proof plan with key lemmas).
2. Semantic Reduction translates the sketch to TIR, with each lemma as a typed node.
3. TIR is lowered to Lean/Coq tactics for formal verification.
4. Failed steps trigger repair feedback; the LLM generates alternative proof strategies.

This approach combines the creative power of LLMs in proof planning with the rigor of formal verification, while the TIR layer provides a structured intermediate for debugging and repair.

6.2 Mathematical Problem Solving

For word problems and competition mathematics:

1. The problem statement is reduced to a TIR specification (variables, constraints, objective).
2. LLM generates a solution strategy in natural language.

3. Each computational step is reduced to TIR and verified via symbolic computation or program execution.

4. Logical steps (e.g., case splits, inequality applications) are verified via SMT or proof assistants.

This ensures that both the arithmetic and the logical structure of the solution are verified, addressing the "correct answer via flawed logic" problem identified in [13].

6.3 AI-assisted Mathematics Education

In educational settings:

1. Student solutions are reduced to TIR, enabling fine-grained error localization.

2. The system can identify *which specific step* is incorrect, rather than just marking the final answer wrong.

3. Verified TIR can be rendered into pedagogical explanations with guaranteed correctness.

4. The system can generate hint sequences by partially reducing the solution and revealing verified intermediate steps.

6.4 Mathematical Research Assistance

For research mathematicians:

1. Conjectures can be formally specified in TIR and checked for consistency.

2. LLM-generated proof ideas can be rapidly prototyped and partially verified.

3. The bidirectional rendering enables communication between formal and informal mathematical discourse.

4. TIR objects can be stored in a searchable knowledge base of verified mathematical results.

7 Experimental Design

We propose a comparative evaluation to validate the semantic reduction framework. While full implementation is beyond the scope of this conceptual paper, we specify the experimental design in detail to enable reproducible validation.

7.1 Datasets

- **GSM8K** [4]: 8,500 grade-school math word problems with step-by-step solutions. Tests basic arithmetic and multi-step reasoning.

- **MATH** [7]: 12,500 competition mathematics problems across 7 subjects and 5 difficulty levels. Tests advanced mathematical reasoning.

- **MiniF2F** [24]: 488 formal olympiad-level problems in Lean and Metamath. Tests formal theorem proving capability.

7.2 Baselines

1. **Pure LLM (CoT)**: Standard few-shot Chain-of-Thought prompting with GPT-4-class model.
2. **Self-Consistency**: CoT with 10-path sampling and majority voting [20].
3. **Program-of-Thought**: LLM generates Python programs executed by interpreter [2].
4. **Tool-augmented**: LLM with access to calculator and symbolic computation tools [18].
5. **TIR-based (ours)**: Full semantic reduction pipeline with TIR and multi-backend verification.

7.3 Metrics

We define three primary metrics that capture different aspects of reliability:

Definition 4 (Accuracy)**.** *The proportion of problems for which the system produces the correct final answer:*

$$Accuracy = \frac{|\{i : \hat{a}_i = a_i\}|}{N} \tag{3}$$

where $\hat{a}_i$ is the predicted answer and a_i is the ground truth.

Definition 5 (Verifiability)**.** *The proportion of reasoning steps that can be successfully reduced to TIR and verified by at least one backend:*

$$Verifiability = \frac{|\{s : V(S(s)) = pass\}|}{|all\ steps|} \tag{4}$$

This metric directly measures the effectiveness of the Semantic Reduction Layer.

Definition 6 (Consistency)**.** *The proportion of problems for which the system produces the same answer across k independent runs (with different random seeds):*

$$Consistency = \frac{|\{i : |\{\hat{a}_{i,j}\}_{j=1}^{k}| = 1\}|}{N} \tag{5}$$

Low consistency indicates probabilistic instability in the reasoning process.

7.4 Hypotheses

H1 The TIR-based system achieves higher *verifiability* than all baselines, because it explicitly structures reasoning for verification.

H2 On problems requiring multi-step logical deduction (MATH hard subset), the TIR-based system achieves higher *accuracy* than baselines, because verification catches logical errors that propagate in pure generation.

H3 The TIR-based system achieves higher *consistency* than pure LLM baselines, because verified reasoning steps are deterministic once reduced.

H4 Program-of-Thought and Tool-augmented baselines achieve high verifiability on computational steps but low verifiability on logical steps, confirming the partial nature of their semantic reduction.

7.5 Expected Results

Table 2 presents our hypothesized results. The TIR-based system is expected to dominate on verifiability and consistency, and to show accuracy improvements particularly on harder problems where logical errors are more prevalent.

Table 2: Hypothesized comparative results. Higher is better for all metrics.

Method	Accuracy	Verifiability	Consistency	Logical Verif.
Pure LLM (CoT)	medium	low	low	none
Self-Consistency	medium-high	low	medium	none
Program-of-Thought	medium-high	medium (computation)	medium	low
Tool-augmented	high	medium (tool-specific)	medium	low
TIR-based (ours)	**high**	**high**	**high**	**high**

8 Discussion

8.1 Limitations

1. *IR design complexity*: Designing a TIR that is expressive enough to cover diverse mathematical domains while remaining amenable to verification is a significant engineering challenge. Different mathematical domains (algebra, geometry, analysis, combinatorics) may require domain-specific TIR extensions.

2. *Semantic mapping cost*: The reduction from natural language to TIR requires sophisticated LLM capabilities and may fail on ambiguous or poorly stated reasoning. The system needs robust error handling and repair mechanisms.

3. *Verification backend coverage*: No single verification backend covers all mathematical domains. The framework requires a multi-backend strategy with intelligent routing, adding system complexity.

4. *Overhead*: The full pipeline (generation, reduction, verification, rendering) incurs significant computational overhead compared to pure generation. This may be acceptable for high-stakes applications but prohibitive for real-time or low-resource settings.

5. *Conceptual paper*: This paper presents a conceptual framework and experimental design. Full implementation and empirical validation remain as future work.

8.2 Extensions

1. *General reasoning systems*: The three-layer architecture is not specific to mathematics. It can be extended to any domain where reasoning reliability matters, such as scientific reasoning, legal argumentation, and software verification.

2. *Agent architectures*: The TIR framework integrates naturally with multi-agent architectures [15]. One agent handles generation, another handles reduction, a third handles verification, and a coordinator manages the repair loop.

3. *Learning to reduce*: The semantic reduction function itself can be learned. By training on pairs of (natural language reasoning, verified TIR), the system can improve reduction accuracy over time, similar to how Lean-STaR [14] interleaves thinking and proving.

4. *Standardization*: A standardized TIR format could enable interoperability between different AI math systems, proof assistants, and mathematical knowledge bases, fostering an ecosystem of verified mathematical reasoning.

8.3 Broader Implications

The semantic reduction framework has implications beyond technical reliability. By making AI mathematical reasoning verifiable at every step, we address growing concerns about the

trustworthiness of AI in mathematics education and research [3]. A system where every reasoning step is both human-interpretable and machine-verifiable enables:

- *Transparent AI*: Users can inspect and understand the reasoning process, rather than treating AI as a black box.
- *Accountable AI*: Errors can be precisely localized to specific reasoning steps, enabling targeted improvement.
- *Collaborative AI*: The bidirectional rendering enables seamless collaboration between humans and AI, with each contributing their strengths.

9 Conclusion

We have presented a three-layer semantic reduction framework for reliable AI mathematics. Our core thesis is that the primary bottleneck in AI-for-Math reliability is not the generation capability of LLMs nor the verification power of formal systems, but the unstable translation between informal natural language and structured formal objects—the *Semantic Reduction Layer*.

Our contributions include: (1) a formal three-layer decomposition of AI mathematical reasoning; (2) the identification and analysis of the semantic reduction bottleneck through theoretical arguments and empirical evidence; (3) the design of a Typed Intermediate Representation (TIR) with type system, operational semantics, and constraint checking; (4) a bidirectional pipeline architecture with Generation $\rightarrow$ TIR $\rightarrow$ Verification $\rightarrow$ Rendering; and (5) a detailed experimental design with accuracy, verifiability, and consistency metrics.

The framework points toward a future of AI mathematics where natural language serves as the fluent interface for human interaction, while a typed, verifiable core ensures that every reasoning step is mathematically sound. This combination of fluency and rigor—of human interpretability and machine verifiability—is essential for building AI systems that mathematicians, educators, and students can truly trust.

References

[1] Kenneth Appel and Wolfgang Haken. Every planar map is four colorable. *Illinois Journal of Mathematics*, 21(3):429–490, 1977. Four-Color Theorem proof with computer assistance.

[2] Wenhu Chen, Xueguang Ma, Xinyi Wang, Ming Zhou, and William Yang Wang. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks, 2022. ICLR 2023 submission.

[3] Xiaoyang Chen and Xiang Jiang. *Lectures on AI for Mathematics*. 1.0 edition, 2026. Course lecture notes covering ML fundamentals, AI theorem proving, conjecture construction, and AI for PDE.

[4] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jakub Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. GSM8K dataset.

[5] DeepSeek-AI et al. DeepSeek-Prover: Advancing theorem proving in LLMs through large-scale synthetic data, 2024.

[6] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models, 2022.

[7] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset, 2021.

[8] William A. Howard. *The Formulae-as-Types Notion of Construction*, pages 479–490. Academic Press, 1980. Curry-Howard correspondence.

[9] Daniel Huang et al. Large language models and mathematical reasoning failures, 2025.

[10] Joy Huang et al. Beyond accuracy: Evaluating the reasoning behavior of large language models – a survey, 2024.

[11] Thomas Hubert et al. Formal mathematics on the verge of a Renaissance. *Nature*, 2025. AlphaProof; IMO silver-medal performance; published online Nov 2025.

[12] Albert Q. Jiang, Wenda Li, and Mateja Jamnik. Multilingual mathematical autoformalization, 2023.

[13] Yao Liang et al. Is your model really a good math reasoner? evaluating mathematical reasoning with checklist, 2024.

[14] Haotian Luo, Jiayu Cheng, Kaiyu Yang, and Anima Anandkumar. Lean-STaR: Learning to interleave thinking and proving, 2024.

[15] Boyuan Ma et al. MA-LoT: Multi-agent Lean-based long chain-of-thought reasoning enhances formal theorem proving, 2025.

[16] OpenAI. Learning to reason with LLMs. OpenAI Blog, 2024. Introduces the o1 reasoning model.

[17] Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving, 2020. Introduces GPT-f for Metamath.

[18] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2302.04761.

[19] Trieu Trinh, Yuhuai Wu, Quoc Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–481, 2024. AlphaGeometry.

[20] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2203.11171.

[21] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022. arXiv:2201.11903.

[22] Yuhuai Wu, Uri Alon, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Christian Szegedy, and Jesse Taylor. Autoformalization with large language models, 2022. NeurIPS 2022.

[23] Kaiyu Yang, Aidan Swope, Alex Gu, Saith Chalamala, Peiyang Song, Xingyao Yu, Mingyue Wang, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2306.15626.

[24] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. minif2f: a cross-system benchmark for formal olympiad-level mathematics, 2022.